LANGCHAIN 1.4+ • CORE FOUNDATION GUIDE

LangChain Core Message Architecture

Mastering SystemMessage, HumanMessage, AIMessage, and ToolMessage in Conversational Agents

An authoritative technical deep dive into LangChain's message polymorphism. Deconstructs role-based conversational state, persona steering, metadata tracing (`name` , `id`), historical message injection, and the multi-turn tool calling handshake.

Target Notebook: `4-messages.ipynb`

Generation Date: November 01, 2026

Language & Runtime: Python 3.14 • Virtualenv (.venv)

Core Framework: LangChain 1.4.7 • Groq (openai/gpt-oss-20b)

B. Learning Objectives

After studying this notebook and study guide, you will master:

- **Core Message Class Polymorphism:** Understand how `BaseMessage` serves as the parent abstraction for `SystemMessage`, `HumanMessage`, `AIMessage`, and `ToolMessage`.
- **Persona Steering via System Prompts:** Direct the model's domain expertise, tone, formatting constraints, and reasoning protocols using `SystemMessage`.
- **Speaker Identification & Observability:** Use optional message metadata attributes (`name` for multi-agent disambiguation and `id` for distributed execution tracing).
- **Context Seeding & Conversation History:** Manually reconstruct multi-turn dialogue histories using synthetic `AIMessage` exemplars.
- **The Two-Turn Tool Handshake:** Deconstruct the exact protocol linking `AIMessage.tool_calls` to `ToolMessage(tool_call_id=...)` for autonomous tool execution.
- **Provider-Agnostic Message Normalization:** Recognize how LangChain bridges native provider message formats across Groq, OpenAI, Anthropic, and Google GenAI.

C. Prerequisites & Setup

> **Environment Specification**
> This notebook runs against the high-throughput **Groq LPU Inference Cloud** executing the `openai/gpt-oss-20b` open-weights reasoning model.

Component	Configuration / Package	Purpose
Python Runtime	`Python >= 3.14`	Modern runtime supporting advanced type hinting and fast async execution.
Virtual Environment	`.venv` (managed via `uv`)	Strictly isolates dependencies defined in `pyproject.toml`.
Environment File	`.env` in workspace root	Securely provides `GROQ_API_KEY` without hardcoding credentials.
Core Dependencies	`langchain>=1.4.7`, `langchain-groq`, `python-dotenv`	Supplies universal chat model interfaces and provider bridge drivers.

> **Security Best Practice**
> Never commit real API keys to version control. Always verify that `.env` is tracked in `.gitignore` and use `.env.example` for team onboarding.

D. Topic Overview: The Architecture of Structured Messages

In simplistic LLM interactions, prompts are passed as plain strings: `model.invoke("Who are you?")`. However, production agentic architectures and conversational applications cannot rely on unstructured strings. Real-world LLM APIs (including OpenAI ChatML, Anthropic Messages API, and Groq) expect a chronological array of distinct message objects, each representing an explicit role and semantic purpose.

LangChain models this reality through a polymorphic hierarchy rooted in `BaseMessage` (located in `langchain_core.messages`). Each message subclass encapsulates:

1. **Role Semantics:** Identifies whether the content originates from the system administrator (`SystemMessage`), the end user (`HumanMessage`), the generative assistant (`AIMessage`), or external code execution (`ToolMessage`).
2. **Structured Payloads:** Allows messages to carry structured metadata beyond plain strings, such as `tool_calls` dictionaries, token usage records, and cryptographic signatures.
3. **Observability Attributes:** Provides native properties including `id` (for LangSmith distributed tracing) and `name` (for differentiating distinct speakers in multi-agent collaboration).

E. Detailed Notebook Walkthrough & Cell Dissection

Cell 0: Markdown Section Header

```
### Message
```

Analysis: Establishes the study module exploring LangChain's message system.

Cell 1: Baseline Invocation & Chat Model Initialization

```
import os
from langchain.chat_models import init_chat_model

os.environ["GROQ_API_KEY"] = os.getenv("GROQ_API_KEY")

model = init_chat_model("groq:openai/gpt-oss-20b")

model.invoke("HI")
```

```
EXECUTION OUTPUT (AIMESSAGE)

AIMessage(content='Hello! How can I help you today?', additional_kwargs={'reasoning_content': 'User says "HI". We should respond politely.'}, response_metadata={'token_usage': {'completion_tokens': 29, 'prompt_tokens': 72, 'total_tokens': 101, 'completion_time': 0.030370052, 'model_name': 'openai/gpt-oss-20b', 'model_provider': 'groq'}})
```

Deconstruction & Key Mechanics:

- `init_chat_model("groq:openai/gpt-oss-20b")`: Instantiates a standardized `ChatGroq` model wrapper under LangChain's unified factory interface.
- **Automatic String Coercion:** Passing the raw string `"HI"` causes LangChain to automatically wrap the string into a single `HumanMessage(content="HI")` before transmitting it to the Groq API.
- **Reasoning Metadata:** In `additional_kwargs['reasoning_content']`, we observe the open-weights model's internal thought deliberation (`User says "HI". We should respond politely.`) generated prior to the user-facing output.

Cell 2: Persona Steering via SystemMessage

```
from langchain.messages import SystemMessage, HumanMessage, AIMessage

messages = [
    SystemMessage("You are a professional football coach"),
    HumanMessage("Give detailed analysis of Kylian Mbappe Playstyle")
]

response = model.invoke(messages)
response.content
```

```
EXECUTION RESULT (EXCERPTS)

## Kylian Mbappé – Play-style Analysis
*(Perspective of a professional football coach – tactical, technical, and developmental insights)*

### 1. Core Physical & Technical Profile
| Attribute | Typical Rating (0–10) | Key Observations |
| Pace & Acceleration | 10/10 | Exceptional burst over 5–15 meters; unmatched top speed... |
| Finishing & Ball Striking | 9.0/10 | Lethal across the goalkeeper; curler into far corner... |
...
```

Architectural Rationale:

- `SystemMessage` instructs the LLM before any user input is processed. It anchors the model's tone, terminology, and evaluation criteria into those of a professional sports coach.
- Notice how the response immediately adopts tactical jargon (e.g., "half-spaces", "low defensive block", "counter-press transition"). Without the `SystemMessage`, the model would produce a generic biographical summary.

Cell 3: Strict Behavioral Guidelines & Comparative Prompting

```
system_msg = SystemMessage("""You are an expert in football tactics.
Always provide examples and explain your reasoning.
Be concise but thorough in you explanations.
""")

human_msg = HumanMessage("who is better in attacking and defence lamine yamal vs kyliam mbappe")

messages = [
    system_msg,
    human_msg
]

response = model.invoke(messages)
print(response.content)
```

```
PRINTED STREAM OUTPUT

**Short answer:**
- **Attacking:** Kylian Mbappé is far superior.
- **Defensive contribution:** Mbappé also edges out Yamal, but Yamal's defensive stats are limited and come from youth-level play.

## 1. Attacking comparison
| Metric | Mbappé (2023-24) | Yamal (2023-24) |
| Goals / 90 min | 0.95 | 0.28 |
...
```

Key Mechanics: Demonstrates multi-line system prompts enforcing constraints: providing concrete examples, structured tables, and concise reasoning.

Cell 4: Message Metadata Attributes (Speaker Disambiguation & Tracing)

```
human_msg = HumanMessage(
    content="Hello",
    name="alice",   # Optional: Identifies the speaker in multi-user/multi-agent chats
    id="msg_123"    # Optional: Unique identifier for application telemetry/tracing
)
```

Deep-Dive on Optional Parameters:

Attribute	Type	Primary Purpose in Production
`name`	`str | None`	Crucial for multi-party conversations (e.g., Group Chats, Multi-Agent Debates). Allows the model to distinguish between inputs from "alice", "bob", or specialized agent nodes ("researcher", "critic").
`id`	`str | None`	Used for distributed tracing (LangSmith, OpenTelemetry) and database synchronization. Allows updating, rating, or referencing specific messages in persistent storage.

Cell 5: Simulating Prior Conversation History

```
# Manual example AI message to seed conversation context
ai_msg = AIMessage("I would be happy to help you with that question")

system_msg = SystemMessage("You are a professional cook, always give detailed ingredients list, and e
xact step wise instructions")

human_msg = HumanMessage("tell me how to cook chicken changezi")

messages = [
    system_msg,
    human_msg,
    ai_msg
]

response = model.invoke(messages)
print(response.content)
```

```
PRINTED OUTPUT

## Chicken Changzi (Changezi Chicken)
*A classic, savory-spicy Mughlai-style chicken dish that showcases the deep umami of rich tomato gravy and
aromatic spices...*

### 1. Ingredients
| Category | Ingredient | Quantity | Notes |
...
```

Pedagogical Insight: Demonstrates how conversational memory is maintained in stateless HTTP APIs. By passing a list of alternating `HumanMessage` and `AIMessage` objects, the application reconstructs the illusion of memory for the LLM.

Cell 6: The Synthetic Tool Calling Handshake

```python
# 1. Model emits a structured tool call request
ai_message = AIMessage(
    content=[],
    tool_calls=[{
        "name": "get_weather",
        "args": {
            "location": "Delhi"
        },
        "id": "call_123"
    }]
)

# 2. Host Python runtime executes tool and obtains result
weather_result = "Sunny, 32 C"

from langchain.messages import ToolMessage

# 3. Wrap tool execution result linked by ID
tool_message = ToolMessage(
    content=weather_result,
    tool_call_id="call_123"  # Must match the call id exactly!
)

# 4. Continue conversation with full context
messages = [
    HumanMessage("What's the weather in Delhi right now?"),
    ai_message,
    tool_message
]

response = model.invoke(messages)
print(response.content)
```

```
FINAL SYNTHESIZED OUTPUT

The weather in Delhi right now is sunny with a temperature of 32°C.
```

The 4-Step Tool Handshake Protocol:

1. The user submits a prompt requiring external live data (`"What's the weather in Delhi?"`).
2. The model determines it needs a tool, producing an `AIMessage` where `content` is empty or minimal, and `tool_calls` contains the function name, arguments, and a unique `id="call_123"` .
3. The Python runtime runs the code and returns the observation wrapped in a `ToolMessage` explicitly referencing `tool_call_id="call_123"` .
4. The entire sequence `[HumanMessage, AIMessage, ToolMessage]` is passed back to the model, allowing it to synthesize a final natural-language response.

F. End-to-End Message Architecture & Handshake Diagram

The diagram below visualizes the class inheritance hierarchy and the exact 4-turn state handshake executed during tool orchestration:

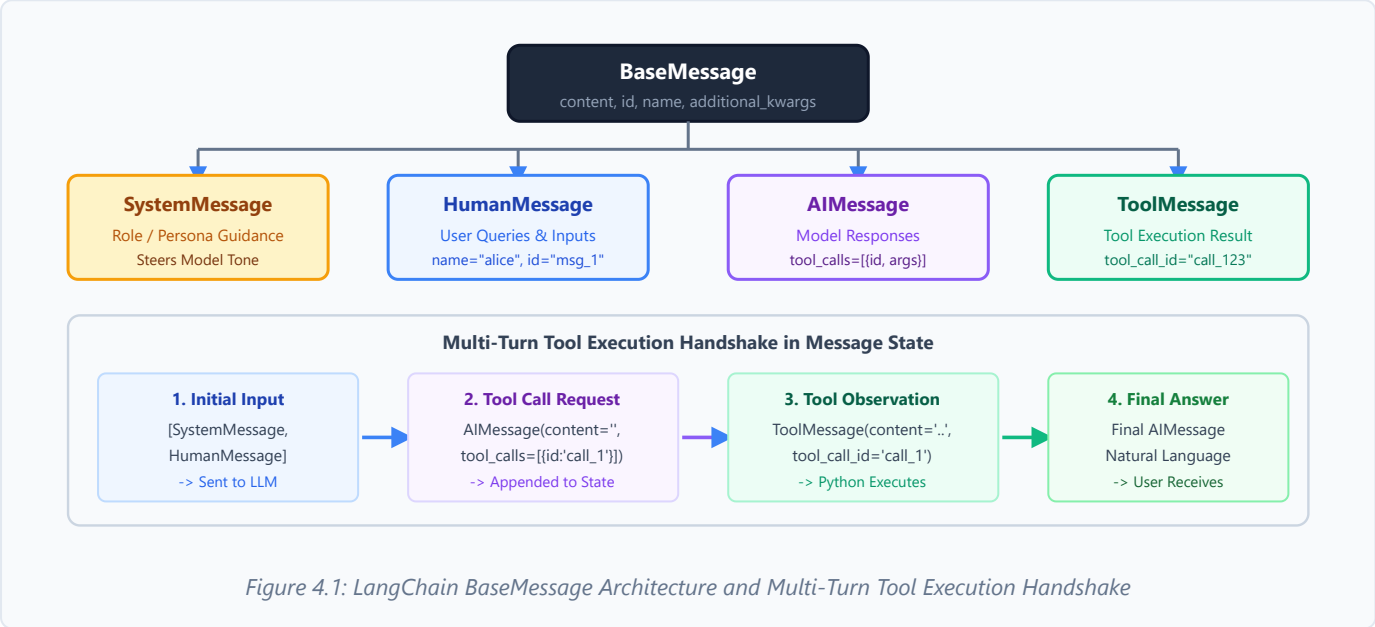


Figure 4.1: LangChain BaseMessage Architecture and Multi-Turn Tool Execution Handshake

G. Important APIs & Classes Reference

Class / Function	Import Path	Key Attributes / Parameters	Description & Role
`BaseMessage`	`langchain_core.messages`	`content` , `id` , `name` , `additional_kwargs`	Root polymorphic message abstraction across all LangChain components.
`SystemMessage`	`langchain.messages`	`content` (instructions / persona)	Sets system-level prompt guidelines, role behaviors, and constraints.
`HumanMessage`	`langchain.messages`	`content` , `name` , `id`	Represents inputs and queries originating from human users.
`AIMessage`	`langchain.messages`	`content` , `tool_calls` , `usage_metadata`	Represents assistant completions or structured tool call requests.
`ToolMessage`	`langchain.messages`	`content` , `tool_call_id` , `name`	Contains the output produced by a local tool, linked via `tool_call_id` .

H. Common Errors, Debugging & Troubleshooting

Trap 1: Mismatched tool_call_id in ToolMessage

Symptom: API returns `400 Bad Request: Invalid tool_call_id` or model produces bizarre hallucinations.

Root Cause: The model emits a tool call with a generated ID (e.g. `id="call_987abc"`). If the application returns a `ToolMessage` with a mismatched ID or hardcoded placeholder, the provider cannot link the observation back to the requesting call.

Fix: Always dynamically copy the ID directly from the tool call object: `ToolMessage(content=result, tool_call_id=tool_call['id'])`.

Trap 2: Passing Raw Strings in a Message List

Symptom: `ValueError: Expected a BaseMessage object, got 'Tell me a joke'`.

Root Cause: While `model.invoke("raw string")` is allowed, passing a list mixing strings and message objects (e.g. `[SystemMessage(...), "raw string"]`) is invalid.

Fix: Ensure all items in a message list are instances of `BaseMessage`: `[SystemMessage(...), HumanMessage(content="...")]`.

I. Practical Example: Stateful Chat History Serializer

In production applications, message histories must be serialized to JSON databases (PostgreSQL, Redis, MongoDB) and restored across web sessions:

```
from langchain_core.messages import (
    messages_to_dict,
    messages_from_dict,
    SystemMessage,
    HumanMessage,
    AIMessage
)

# 1. Active conversation state in memory
history = [
    SystemMessage(content="You are a helpful coding tutor."),
    HumanMessage(content="What is Python list comprehension?", id="req_001"),
    AIMessage(content="It's a concise way to create lists: [x for x in iterable].", id="res_001")
]

# 2. Serialize to standard JSON for database storage
serialized_json = messages_to_dict(history)

# 3. Deserialize back into strongly-typed LangChain objects upon next user turn
restored_history = messages_from_dict(serialized_json)
assert isinstance(restored_history[0], SystemMessage)
assert restored_history[1].id == "req_001"
```

J. Key Takeaways & Conceptual Map

- **Messages Replace Strings:** Professional LangChain development operates on sequences of `BaseMessage` objects, not plain strings.

- **SystemMessage Shapes the Persona:** Use system messages to constrain output formats, adopt specialized domains, and establish boundaries.
- **Metadata Enables Enterprise Tracing:** Use `id` for observability and `name` for speaker disambiguation in multi-agent environments.
- **Tool Calling is a Handshake:** An `AIMessage` requests a tool via `tool_calls`; the host environment executes the tool and responds with a `ToolMessage` matching the exact `tool_call_id`.

K. Practice Exercises & Knowledge Checks

Question 1 (Conceptual): Why does ToolMessage require a tool_call_id attribute?

Why can't LangChain just assume the latest tool result matches the latest user question?

> **Answer:** Modern LLMs support *parallel tool calling*, meaning a single model response can emit multiple tool calls simultaneously (e.g., fetching weather for Delhi, Tokyo, and London in one turn). The `tool_call_id` is the cryptographic foreign key that pairs each observation precisely with its specific request.

Question 2 (Code Tracing): In Cell 6, why is AIMessage initialized with content=[]?

Does the model output text when making a tool call?

> **Answer:** When an LLM decides to invoke a tool, its generative budget is directed entirely into formulating the structured tool arguments inside `tool_calls`. It intentionally leaves conversational `content` empty (or as an empty list) because it must await the tool observation before generating natural-language text.

Question 3 (Architecture): How does the name attribute on HumanMessage prevent multi-agent confusion?

> **Answer:** In multi-agent architectures (or group chat systems), multiple participants write to the same shared message list. Attaching `name="researcher"` or `name="critic"` allows the downstream model to understand who said what without corrupting the message text itself.

L. Further Learning & Official Documentation

- LangChain Messages Concept Guide
- Groq Cloud Model Documentation
- Next study guide in sequence: **5-structure.pdf** (Structured Outputs & Pydantic Schemas).